GUÍA DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Plano interactivo de un ambiente con Canvas en Jetpack Compose				
NÚMERO DE PRÁCTICA:	7	**AÑO LECTIVO:**	2026B	**NRO. SEMESTRE:**	VIII
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**X**	**MÁXIMO DE ESTUDIANTES**		2
FECHA INICIO:	13/10/2026	**FECHA FIN:**	17/10/2026	**DURACIÓN:**	100 minutos
RECURSOS A UTILIZAR: Android Studio (Quail o superior), Kotlin, Jetpack Compose, emulador Android o dispositivo físico (API 34 o superior)					
DOCENTE(s): Roxana Evelyn Limache Calatayud					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: - Comprender el Composable Canvas y las funciones de dibujo (DrawScope) disponibles en Jetpack Compose. - Implementar el plano de un ambiente dibujando sus salas mediante primitivas de Canvas y marcadores interactivos. - Detectar toques sobre regiones específicas del Canvas y mostrar información adicional al seleccionar un marcador.	
TEMAS: - Canvas y DrawScope en Jetpack Compose - Primitivas de dibujo: drawRect, drawLine, drawCircle - Detección de toques sobre un Canvas (pointerInput, detectTapGestures)	
COMPETENCIAS	C.a Aplica de forma flexible técnicas, métodos, principios, normas, estándares y herramientas de ingeniería necesarias para la construcción de software e implementación de sistemas de información.
	C.b Investiga nuevos modelos, metodologías, técnicas, herramientas y tecnologías por ser necesarias para mantener la vigencia en el desempeño profesional.
	C.c
	C.d

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

Revisar el siguiente recurso antes de la sesión:

- Video: "Canvas Basics in Jetpack Compose"
https://www.youtube.com/watch?v=cCIPYW_N12o

- Android Developers, gráficos en Compose:
https://developer.android.com/develop/ui/compose/graphics/draw/overview

De los componentes UI personalizados clásicos al Canvas de Compose

En el enfoque clásico de Views, un componente de dibujo personalizado (por ejemplo el plano de un ambiente) se implementaba creando una subclase de View, sobrescribiendo onDraw(canvas: Canvas) para pintar con la API de android.graphics.Canvas, y sobrescribiendo onTouchEvent(event: MotionEvent) para detectar toques manualmente sobre coordenadas de pantalla. En Jetpack Compose no se necesita ninguna subclase: existe un Composable llamado Canvas que recibe un modifier y un lambda de dibujo, como se resume en la Figura 1.

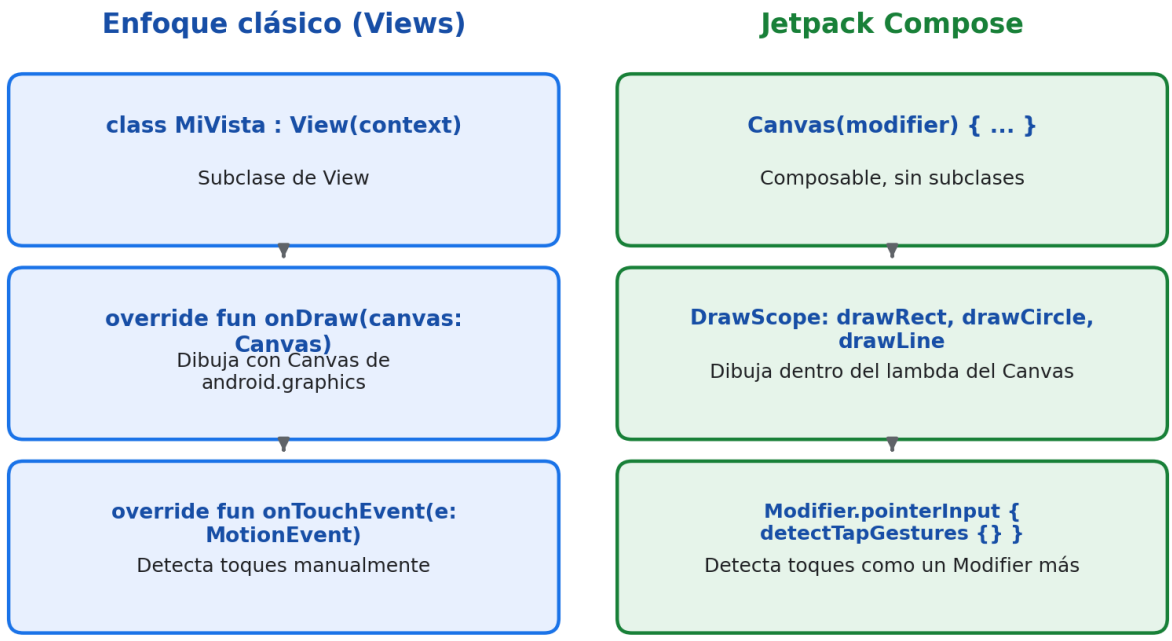


Figura 1. Del enfoque clásico de Views al Canvas Composable de Jetpack Compose (elaboración propia).

Canvas y DrawScope

Dentro del lambda de Canvas { ... }, el receptor es un DrawScope que expone funciones de dibujo como drawRect (rectángulos, útiles para representar ambientes o salones), drawLine (para las paredes divisorias) y drawCircle (para marcadores puntuales). Cada función recibe un color y parámetros de posición y tamaño expresados como Offset y Size, en un sistema de coordenadas donde el origen (0, 0) está en la esquina superior izquierda del Canvas.

Detección de toques sobre un Canvas

A diferencia de un botón o una tarjeta, el Canvas no tiene "hijos" sobre los que Compose pueda detectar clics automáticamente: todo el Canvas es una sola superficie de dibujo. Para saber si el

usuario tocó un marcador específico, se agrega Modifier.pointerInput(Unit) { detectTapGestures { offset -> ... } }, que entrega la posición exacta del toque, y luego se compara manualmente esa posición contra el centro y el radio de cada marcador dibujado, actualizando el estado y mostrando un diálogo si corresponde, como se resume en la Figura 2.

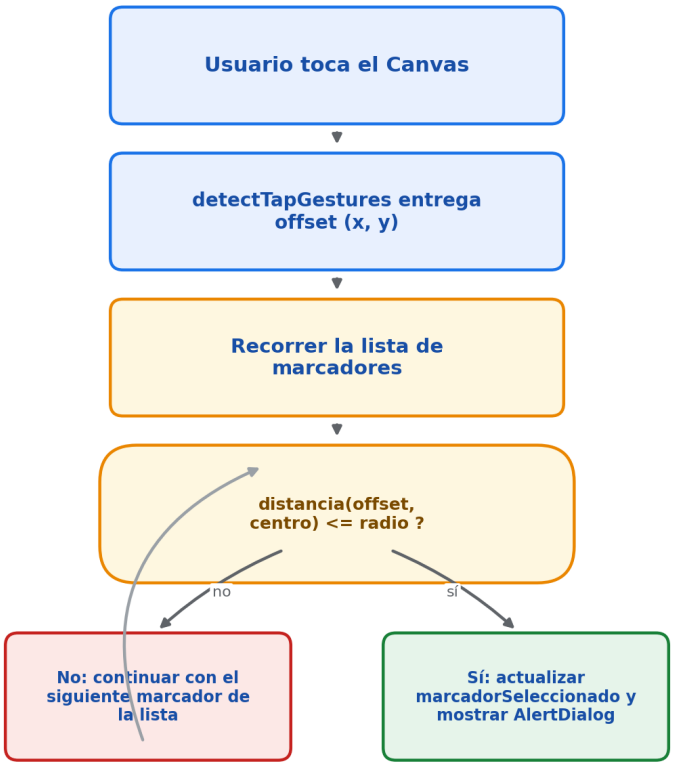


Figura 2. Flujo de detección de un toque sobre un marcador dibujado en el Canvas (elaboración propia).

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

A continuación se detallan, paso a paso, las instrucciones para dibujar el plano de un ambiente con dos salones y marcadores interactivos que muestran información al ser tocados. También puede apoyarse en el siguiente video tutorial:

Video de referencia: "Canvas Basics in Jetpack Compose"
https://www.youtube.com/watch?v=cCIPYW_N12o

Paso 1. Crear un nuevo proyecto.

nombre sugerido para el proyecto: "PlanoInteractivo_Compose", Package name: com.example.planointeractivo_compose, Language: Kotlin, Minimum SDK: el que venga por defecto en la plantilla (recuerden usar un emulador o dispositivo con API 34 o superior, según la sección de Recursos a utilizar).

Paso 2. Agregar, debajo del package y antes de la clase MainActivity, todas las importaciones que se usarán en este ejercicio:

Código (importaciones):

```
import android.os.Bundle
import androidx.activity.ComponentActivity
```

```
import androidx.activity.compose.setContent
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.gestures.detectTapGestures
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.planointeractivo_compose.ui.theme.PlanoInteractivo_ComposeTheme
```

Paso 3. Definir, fuera de cualquier Composable, la data class que representará cada marcador del plano:

Código (data class Marcador):

```
data class Marcador(
    val offset: Offset,
    val radio: Float,
    val titulo: String,
    val descripcion: String
)
```

Paso 4. Crear el Composable PlanoAmbiente, con una lista de al menos 2 marcadores de muestra (remember) y una variable de estado para el marcador seleccionado. Por ahora dejamos el resto del cuerpo vacío; lo iremos completando en los siguientes pasos:

Código (PlanoAmbiente – Paso 4):

```
@Composable
fun PlanoAmbiente() {
    val marcadores = remember {
        listOf(
            Marcador(Offset(100f, 150f), 40f, "Sala A", "Sala de cómputo 1"),
            Marcador(Offset(250f, 150f), 40f, "Sala B", "Sala de cómputo 2")
        )
    }
    var marcadorSeleccionado by remember { mutableStateOf<Marcador?>(null) }

    // el Canvas se agrega en el Paso 5
}
```

Paso 5. Declarar un Canvas que ocupe el ancho disponible con una altura fija, agregando ya el Modifier.pointerInput con detectTapGestures (todavía vacío). Dentro del Canvas dejamos, por ahora, un comentario donde irá el dibujo:

Código (PlanoAmbiente – Paso 5):

```
@Composable
fun PlanoAmbiente() {
    val marcadores = remember {
        listOf(
            Marcador(Offset(100f, 150f), 40f, "Sala A", "Sala de cómputo 1"),
            Marcador(Offset(250f, 150f), 40f, "Sala B", "Sala de cómputo 2")
        )
    }
    var marcadorSeleccionado by remember { mutableStateOf<Marcador?>(null) }

    Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(300.dp)
            .pointerInput(Unit) {
                detectTapGestures { offset ->
                    // aquí se calcula el marcador tocado, en el Paso 8
                }
            }
    ) {
        // aquí se dibuja el plano, en los Pasos 6 y 7
    }
}
```

Paso 6. Dentro del Canvas, dibujar los dos salones con drawRect y la pared divisoria entre ellos con drawLine:

Código (PlanoAmbiente – Paso 6):

```
    Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(300.dp)
            .pointerInput(Unit) {
                detectTapGestures { offset ->
                    // aquí se calcula el marcador tocado, en el Paso 8
                }
            }
    ) {
        // Salón 1
        drawRect(
            color = Color(0xFFBBDEFB),
            topLeft = Offset(20f, 20f),
            size = Size(150f, 250f)
        )
        // Salón 2
        drawRect(
            color = Color(0xFFC8E6C9),
            topLeft = Offset(180f, 20f),
            size = Size(150f, 250f)
        )
        // Pared divisoria
        drawLine(
            color = Color.DarkGray,
            start = Offset(170f, 20f),
            end = Offset(170f, 270f),
            strokeWidth = 4f
        )
        // aquí se dibujan los marcadores, en el Paso 7
    }
```

Paso 7. Dentro del mismo Canvas, recorrer la lista de marcadores dibujando cada uno con drawCircle. Con esto, el dibujo del plano queda completo:

Código (PlanoAmbiente – Paso 7):

```
        // Pared divisoria
        drawLine(
            color = Color.DarkGray,
            start = Offset(170f, 20f),
            end = Offset(170f, 270f),
            strokeWidth = 4f
        )
        marcadores.forEach { marcador ->
            drawCircle(
                color = Color.Red,
                radius = marcador.radio,
                center = marcador.offset
            )
        }
    }
```

Paso 8. Dentro de detectTapGestures, recorrer la lista de marcadores y calcular la distancia entre el toque y el centro de cada marcador; si es menor o igual al radio, asignarlo a marcadorSeleccionado. Con esto, el Canvas queda completo:

Código (PlanoAmbiente – Paso 8):

```
            .pointerInput(Unit) {
                detectTapGestures { offset ->
                    marcadores.forEach { marcador ->
                        if ((offset - marcador.offset).getDistance() <= marcador.radio) {
                            marcadorSeleccionado = marcador
                        }
                    }
                }
            }
```

Paso 9. Fuera del Canvas, mostrar un AlertDialog cuando marcadorSeleccionado no sea null, con el título y la descripción del marcador. Con esto, PlanoAmbiente queda completo:

Código (PlanoAmbiente completo):

```
@Composable
fun PlanoAmbiente() {
    val marcadores = remember {
        listOf(
            Marcador(Offset(100f, 150f), 40f, "Sala A", "Sala de cómputo 1"),
            Marcador(Offset(250f, 150f), 40f, "Sala B", "Sala de cómputo 2")
        )
    }
    var marcadorSeleccionado by remember { mutableStateOf<Marcador?>(null) }

    Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(300.dp)
            .pointerInput(Unit) {
                detectTapGestures { offset ->
                    marcadores.forEach { marcador ->
```

```
                        if ((offset - marcador.offset).getDistance() <= marcador.radio) {
                            marcadorSeleccionado = marcador
                        }
                    }
                }
            }
    ) {
        // Salón 1
        drawRect(
            color = Color(0xFFBBDEFB),
            topLeft = Offset(20f, 20f),
            size = Size(150f, 250f)
        )
        // Salón 2
        drawRect(
            color = Color(0xFFC8E6C9),
            topLeft = Offset(180f, 20f),
            size = Size(150f, 250f)
        )
        // Pared divisoria
        drawLine(
            color = Color.DarkGray,
            start = Offset(170f, 20f),
            end = Offset(170f, 270f),
            strokeWidth = 4f
        )
        marcadores.forEach { marcador ->
            drawCircle(
                color = Color.Red,
                radius = marcador.radio,
                center = marcador.offset
            )
        }
    }

    marcadorSeleccionado?.let { marcador ->
        AlertDialog(
            onDismissRequest = { marcadorSeleccionado = null },
            title = { Text(marcador.titulo) },
            text = { Text(marcador.descripcion) },
            confirmButton = {
                TextButton(onClick = { marcadorSeleccionado = null }) {
                    Text("Cerrar")
                }
            }
        )
    }
}
```

Paso 10. Dentro de MainActivity, en el bloque setContent, mostrar PlanoAmbiente:

Código (MainActivity.kt completo):

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            PlanoInteractivo_ComposeTheme {
                Surface(modifier = Modifier.fillMaxSize()) {
                    PlanoAmbiente()
                }
            }
```

```
        }
    }
}

@Preview(showBackground = true)
@Composable
fun PlanoAmbientePreview() {
    PlanoInteractivo_ComposeTheme {
        PlanoAmbiente()
    }
}
```

Paso 11. Ejecutar la aplicación en un emulador y verificar que, al tocar cada marcador dibujado sobre el plano, se abre el diálogo correspondiente con su información.

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Amplía el plano resuelto por el docente para representar la visualización interior de al menos una edificación del campus:

1. Amplía la lista de marcadores y agrega los rectángulos/líneas necesarios para representar al menos 4 ambientes (salones y/o patios) de una edificación real del campus, cada uno con su etiqueta visible en el plano.
2. Convierte la lista de marcadores en datos cargados desde un archivo (por ejemplo un .json o .csv en assets/) en lugar de estar escritos directamente en el código, siguiendo un patrón de persistencia similar al usado en el Laboratorio 03.
3. Traslada el estado del plano y sus marcadores a un ViewModel (PlanoViewModel), siguiendo el patrón de ViewModel visto en el Laboratorio 05, en lugar de mantenerlo directamente en el Composable.
4. Integra este plano como una nueva pantalla accesible desde MapaScreen (Laboratorio 05) o desde una tarjeta del Laboratorio 06.

Entregable: informe con capturas del plano con al menos 4 ambientes, del diálogo de información al pulsar cada marcador, y una breve descripción de cómo se cargan los datos desde el archivo.

IV. CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (plano de un ambiente dibujado con Canvas, donde tocar un marcador abre un diálogo con su información)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.
2. En tu propia implementación, ¿cómo calculaste si un toque cayó dentro de un marcador circular? ¿Qué otra estrategia se te ocurre para detectar toques sobre formas no circulares, como un rectángulo o un polígono?
3. En forma individual, comenta en el video de referencia (https://www.youtube.com/watch?v=cCIPYW_N12o) qué otra forma se te ocurre para lograr un dibujo interactivo similar sin depender directamente de Canvas y DrawScope. (Colocar la captura del comentario; uno por integrante del grupo).

V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

[1] Android Developers, "Graphics in Compose". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/graphics/draw/overview

[2] Android Developers, "Pointer input in Compose". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/touch-input/pointer-input

[3] "Canvas Basics in Jetpack Compose - Android Studio Tutorial", video. [En línea]. Disponible: https://www.youtube.com/watch?v=cCIPYW_N12o

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:	**INSTRUMENTOS:**
Ejercicios propuestos	Rúbrica de evaluación

CRITERIOS DE EVALUACIÓN

- Cumplimiento de los objetivos y ejercicios propuestos en la guía.
- Correcta aplicación de los conceptos y herramientas del tema desarrollado.
- Funcionamiento correcto de lo implementado (compilación/ejecución sin errores).
- Informe (formato estudiante) completo, con evidencias y cuestionario resuelto.
- Entrega dentro del plazo establecido.
- Se permite el uso de Inteligencia Artificial (IA) como fuente de consulta o apoyo para resolver los ejercicios; sin embargo, el desarrollo completo del laboratorio no puede basarse únicamente en respuestas generadas por IA. El estudiante debe evidenciar comprensión propia mediante la justificación razonada de sus respuestas, especialmente en el cuestionario.